

Práctica 4: Desarrollo del Front-End del Ecommerce con Angular

Curso: Desarrollo Web Moderno

Proyecto Continuoado: TechStore Online con Angular

Importante: Esta práctica reemplaza el front-end desarrollado con HTML, CSS y JavaScript puro por una aplicación desarrollada con **Angular**. El objetivo es transformar el Ecommerce en una **Single Page Application (SPA)** moderna.

Objetivo General

Migrar el Ecommerce desarrollado anteriormente hacia Angular, aplicando componentes, servicios, enrutamiento, formularios reactivos y consumo de APIs.

Objetivos de Aprendizaje

Al finalizar la práctica, el estudiante será capaz de:

- Crear aplicaciones con Angular CLI.
 - Diseñar componentes reutilizables.
 - Implementar navegación SPA.
 - Trabajar con Data Binding.
 - Utilizar Directivas.
 - Crear Servicios.
 - Consumir APIs REST.
 - Implementar formularios reactivos.
 - Aplicar Pipes.
 - Utilizar RxJS y Observables.
 - Administrar estado de la aplicación.
-

Tecnologías a utilizar

- Angular 20+
 - TypeScript
 - RxJS
 - Angular Router
 - Angular Forms
 - Bootstrap o Angular Material
-

Escenario del Proyecto

La empresa TechStore desea modernizar su plataforma para mejorar la experiencia del usuario y facilitar el mantenimiento del sistema.

El Ecommerce debe funcionar completamente sin recargar la página.

Configuración Inicial

Crear el proyecto:

```
ng new techstore  
cd techstore  
ng serve -o
```

Instalar Bootstrap:

```
npm install bootstrap
```

Agregar Bootstrap en:

angular.json

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "src/styles.css"  
]
```

Arquitectura del Proyecto

```
src/
├── app/
│   ├── components/
│   ├── pages/
│   │   ├── home/
│   │   ├── productos/
│   │   ├── detalle-producto/
│   │   ├── carrito/
│   │   ├── checkout/
│   │   ├── contacto/
│   │   └── not-found/
│   ├── services/
│   │   ├── producto.service.ts
│   │   ├── carrito.service.ts
│   │   └── auth.service.ts
│   ├── models/
│   │   ├── producto.ts
│   │   ├── cliente.ts
│   │   └── orden.ts
│   ├── guards/
│   ├── interceptors/
│   ├── pipes/
│   └── shared/
└── assets/
```

Requerimientos Funcionales

1. Componentes Obligatorios

Crear los siguientes componentes:

```
ng generate component components/navbar
ng generate component components/footer
ng generate component components/product-card
ng generate component components/search-bar
ng generate component components/category-menu
```

Páginas:

```
ng generate component pages/home
ng generate component pages/productos
ng generate component pages/detalle-producto
ng generate component pages/carrito
ng generate component pages/checkout
ng generate component pages/contacto
ng generate component pages/not-found
```

2. Routing

Configurar rutas:

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'productos', component: ProductosComponent },
  { path: 'producto/:id', component: DetalleProductoComponent },
  { path: 'carrito', component: CarritoComponent },
  { path: 'checkout', component: CheckoutComponent },
  { path: 'contacto', component: ContactoComponent },
  { path: '**', component: NotFoundComponent }
];
```

Implementar:

```
<router-outlet></router-outlet>
```

3. Data Binding

Aplicar los cuatro tipos:

Interpolación

```
{{ producto.nombre }}
```

Property Binding

```
<img [src]="producto.imagen">
```

Event Binding

```
<button (click)="agregarCarrito()">
```

Two-way Binding

```
<input [(ngModel)]="busqueda">
```

4. Directivas

Usar:

Estructurales

*ngIf

*ngFor

*ngSwitch

Ejemplo:

```
<div *ngFor="let producto of productos">
```

De atributo

[ngClass]

[ngStyle]

5. Servicios

Crear:

ProductoService

Funciones:

obtenerProductos()

obtenerProducto(id)

buscarProducto()

CarritoService

Funciones:

agregar()
eliminar()
vaciar()
calcularTotal()

6. Comunicación entre Componentes

Implementar:

@Input()
@Output()
EventEmitter

Ejemplo:

@Input() producto!: Producto;

@Output()
agregado = new EventEmitter();

7. Formularios Reactivos

Implementar formulario de checkout:

Campos:

- Nombre
- Email
- Dirección
- Teléfono
- Tarjeta

Validaciones:

Validators.required
Validators.email
Validators.minLength
Validators.pattern

Mostrar errores dinámicos.

8. Consumo de API REST

Consumir una API pública:

Ejemplo:

- Fake Store API

Usar:

HttpClient
Observable
subscribe()

Ejemplo:

this.http.get(url)

9. RxJS

Utilizar:

Observable
BehaviorSubject
map
filter
tap
catchError

El carrito debe actualizarse en tiempo real.

10. Pipes

Crear Pipe personalizado:

ng generate pipe pipes/descuento

Ejemplo:

{{ precio | descuento:20 }}

Usar también:

- CurrencyPipe
 - DatePipe
 - UpperCasePipe
-

11. LocalStorage

Persistir:

- Carrito
 - Tema oscuro
 - Preferencias
-

12. Guards

Crear:

ng generate guard guards/auth

Proteger:

checkout
perfil
historial

13. Interceptors

Crear interceptor para:

- Token JWT
- Manejo global de errores

ng generate interceptor interceptors/auth

14. Lazy Loading

Configurar módulos cargados bajo demanda:


```
loadChildren: () =>
import('./productos/productos.module')
```

15. Página de Administración

Crear panel administrativo:

Funciones:

- Agregar producto
 - Editar producto
 - Eliminar producto
 - Gestionar inventario
-

Funcionalidades Obligatorias del Ecommerce

Catálogo

- Mostrar productos dinámicamente.
- Búsqueda en tiempo real.
- Filtros por categoría y precio.

Carrito

- Agregar productos.
- Eliminar productos.
- Actualizar cantidades.
- Persistencia con LocalStorage.

Checkout

- Generar orden.
 - Calcular impuestos.
 - Mostrar resumen de compra.
-

Conceptos Angular Evaluables

Los estudiantes deben demostrar el uso de:

Componentes
Templates
Data Binding
Directivas
Servicios
Routing
Formularios Reactivos
Pipes
Observables
RxJS
HttpClient
Guards
Interceptors
Lazy Loading
LocalStorage

Funcionalidades Bonus (+20 puntos)

Implementar al menos tres:

- Autenticación JWT
 - Login y Registro
 - Lista de favoritos
 - Tema oscuro
 - Historial de compras
 - Paginación
 - Infinite Scroll
 - Integración con Stripe o PayPal
 - Internacionalización (i18n)
-

Entregables

1. Proyecto Angular completo.
2. Código fuente en GitHub.
3. Video demostrativo (5–10 minutos).
4. Documento técnico PDF explicando:

- Arquitectura.
 - Componentes.
 - Servicios.
 - Rutas.
 - Observables utilizados.
-

Rúbrica de Evaluación (100 puntos)

Criterio	Puntos
Componentes y Routing	20
Servicios y APIs	15
Formularios Reactivos	15
RxJS y Observables	15
Carrito y LocalStorage	10
Guards e Interceptors	10
Lazy Loading	5
Calidad del código	5
Diseño y UX	5

Resultado Esperado

Al finalizar las cuatro prácticas, los estudiantes habrán construido un Ecommerce moderno con:

- ✓ HTML5 semántico
- ✓ CSS3 responsivo
- ✓ JavaScript ES6+
- ✓ Angular SPA
- ✓ APIs REST
- ✓ RxJS y Observables
- ✓ Arquitectura profesional Front-End

Este proyecto constituye un **portafolio profesional de nivel Junior Front-End**, alineado con las competencias requeridas por el mercado actual.